Experiment No. 1
Cryptography in Blockchain: Merkle Root (Tree Hash)

Aim

To study cryptography in blockchain (SHA-256 hashing, nonce and Proof-of-Work) and to compute a Merkle root tree hash.

Theory

Cryptographic hash function. It takes an input of arbitrary length and produces a fixed-length output called a hash or digest. In blockchain, SHA-256 (Secure Hash Algorithm, 256-bit) is widely used. Key properties:

- **Determinism** – the same input always gives the same output.
- **Avalanche effect** – a tiny change in input drastically changes the hash.
- **Pre-image resistance** – the input cannot be recovered from the hash.
- **Collision resistance** – no two different inputs produce the same hash.

These properties make hashes ideal for ensuring data integrity across a distributed network.

Nonce and Proof-of-Work. A nonce (number used once) is an integer that miners repeatedly increment and combine with the block data to produce a hash that satisfies the network's difficulty target, typically a hash beginning with a certain number of leading zeros. This target hash is called the *Golden Nonce*. Searching for it is computationally expensive by design, which is the foundation of the Proof-of-Work consensus used in Bitcoin and makes tampering prohibitively costly.

Merkle tree. A Merkle tree is a binary tree of hashes. Each leaf holds the hash of one transaction; each parent holds the hash of the concatenation of its two child hashes. This repeats level by level until a single hash remains: the **Merkle root**. If a level has an odd number of nodes, the last node is duplicated before pairing. The Merkle root is stored in the block header as a compact cryptographic summary of all transactions in the block.

How it works (transactions T1 to T5):

1. Each transaction string is hashed individually, giving leaf hashes H1 to H5.
2. H5 is duplicated to make the count even: H1–H2, H3–H4 and H5–H5 are combined.
3. The three resulting hashes are paired again (duplicating the odd one out) and hashed.
4. This continues until one final hash remains: the Merkle root.

Benefits:

- **Efficient verification** – a single transaction can be verified with a Merkle proof, without downloading the full block.

- **Tamper detection** – altering a transaction changes its leaf hash and propagates up to the root.
- **Bandwidth savings** – light clients (SPV nodes) need only the root hash and a small proof path.

Use cases: Bitcoin and Ethereum (summarizing block transactions), Git (Merkle DAG), certificate transparency logs, and IPFS (content-addressed Merkle DAGs).

Requirements

- Python 3 (standard library only: `hashlib`).
- Terminal or IDE to run the programs.

Procedure

Program 1: SHA-256 hash of a string

```
import hashlib                                  # fixed

def create_hash(string):

   hash_object = hashlib.sha256()

   hash_object.update(string.encode('utf-8'))

   hash_string = hash_object.hexdigest()              # fixed

   return hash_string

input_string = input("Enter a string: ")
hash_result = create_hash(input_string)

print("Hash:", hash_result)              # fixed
```

Output:

```
Enter a string: Yash
Hash: 48a534e64e3b91484555483f2b6bea7069261bf6b2e363b69b9bdcf2d15684fa
```

Program 2: Hash of a string combined with a nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
```

```
# Concatenate the string and nonce
hash_string = input_string + nonce    # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode("utf-8"))

hash_code = hash_object.hexdigest()
# Print the hash code
print("Hash Code:", hash_code)   # Error
```

Output:

```
Enter a string: yash
Enter the nonce: 9
Hash Code: d09e81445859b10bce49fc4b1a5ca8afaf59b102e0dffd27576e386e578ba184
```

Program 3: Finding the nonce (Golden Nonce)

```
import hashlib

def find_nonce(input_string, num_zeros):
  nonce = 0
  hash_prefix = "0" * num_zeros

  while True:
      # Concatenate the string and nonce
      hash_string = input_string + str(nonce)   # Error
      # Calculate the hash using SHA-256
      hash_object = hashlib.sha256(hash_string.encode("utf-8"))
      hash_code = hash_object.hexdigest()

      # Check if the hash code has the required number of leading
    zeros
      if hash_code.startswith(hash_prefix):   # Error
          print("Hash:", hash_code)
          return nonce

      nonce += 1

# Get user input
input_string = "A"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)   # Error
```

Output:

```
Hash: 06663975f75f189f1d70bd14d8f22df264cd6a5c7575d6875183d0ec76432fbb
Input String: A
Leading Zeros: 1
Expected Nonce: 9
```

Program 4: Merkle root of five transactions

```
import hashlib
def build_merkle_tree(transactions):   # fixed

  if len(transactions) == 0:
      return None

  if len(transactions) == 1:
      return transactions[0]

  while len(transactions) > 1:
      if len(transactions) % 2 != 0:
          transactions.append(transactions[-1])

      new_transactions = []
      for i in range(0, len(transactions), 2):
          combined = transactions[i] + transactions[i + 1]
          hash_combined = hashlib.sha256(combined.encode("utf-8")
   ).hexdigest()
          new_transactions.append(hash_combined)   # fixed

      transactions = new_transactions

  return transactions[0]

transactions = [
    "Transaction 1",
    "Transaction 2",
    "Transaction 3",
    "Transaction 4",
    "Transaction 5",
]

merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)   # fixed
```

Output:

```
Merkle Root: a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077
```

Output

SHA-256 hashes were generated for a string with and without a nonce, a nonce giving a hash with one leading zero was found (nonce 9 for the input "A"), and the Merkle root of five transactions was computed.

Conclusion

Cryptographic hashing, nonce search and Merkle root construction were implemented in Python using SHA-256.

Experiment completed successfully.